Projet gestionyobs

Journal du 21 octobre 2026

Ce document décrit ce qui a été fait aujourd'hui sur le logiciel de gestion pour quincaillerie (Flutter + SQLite), l'état actuel du projet et le plan pour reprendre demain. Il complète le premier récapitulatif (Recapitulatif_Projet_gestionyobs.pdf).

1. Résumé de la journée

- Configuration de l'environnement Flutter sur Windows (Visual Studio, doctor) et création du projet **gestionyobs**.
- Page de connexion avec le logo YOBS, validation des champs et oeil du mot de passe.
- Fichier de configuration lu à chaque démarrage, avec assistant au premier lancement (cas 1 : SQLite locale).
- Base SQLite : utilisateurs avec mots de passe hachés, premier administrateur, connexion réelle.
- **Agences** : création, choix après la connexion, description (zone de texte), migration de la base en version 3.
- **Utilisateurs** : liste, création, affiliation aux agences, rôles vendeur / comptable / magasinier / administrateur.
- **Modification d'un utilisateur** (étape 8) : nom, rôle, agences, désactivation, réinitialisation du mot de passe, avec règles de sécurité.
- **Conception du système de droits** : l'administrateur choisira lui-même ce que chaque rôle peut faire (à construire demain).

2. Fonctionnement actuel du logiciel

```
Démarrage -> lecture de config.json -> ouverture de la base SQLite
   (absent ou erreur : assistant de configuration)
        |
        v
Aucun utilisateur ? -> création de l'administrateur
        |
        v
Page de connexion (identifiant + mot de passe haché)
        |
        v
ApresConnexionPage : quelles agences pour cet utilisateur ?
   - aucune agence et admin    -> créer la première agence
   - non-admin, une seule      -> entrée directe dans l'agence
   - sinon                     -> liste des agences à choisir
        |
        v
Accueil (provisoire) : utilisateur, rôle, agence
   admin : bouton Utilisateurs -> liste -> créer / modifier un utilisateur
```

3. Étapes réalisées

Étape	Contenu	État
Préparation	Flutter, Visual Studio, projet gestionyobs, logo, page de connexion de base	Validé
1 à 5	Fichier de configuration, assistant, logique de démarrage, base SQLite, connexion réelle	Validé
6	Agences (création, choix), colonne description, migration en version 3	Validé
7	Liste et création des utilisateurs, affiliation aux agences, rôles supplémentaires	Validé
8	Modification d'un utilisateur, désactivation, réinitialisation du mot de passe	Code fourni, tests à confirmer
9 (prévu)	Droits par rôle réglables par l'administrateur, menu selon les droits	À faire (conception terminée)

4. Utilisateurs, rôles et agences

Rôles actuels

Code (base)	Libellé affiché	Remarque
`vendeur`	Vendeur	Doit être affilié à des agences
`comptable`	Comptable	Doit être affilié à des agences
`magasinier`	Magasinier	Doit être affilié à des agences
`admin`	Administrateur	Accès à toutes les agences, aucune affiliation nécessaire

La liste des rôles est définie à un seul endroit (rolesDisponibles dans utilisateur.dart), et libelleRole() donne le texte à afficher. Le code du rôle est stocké dans la base et ne change jamais ; le libellé peut être modifié librement. Pour l'instant, comptable et magasinier se comportent comme un vendeur : seuls leurs droits propres manquent (étape 9).

Règles d'accès aux agences

Cas	Comportement
Aucune agence, utilisateur admin	Page « Créer la première agence », puis accueil
Admin, des agences existent	Il voit toutes les agences et peut en créer
Non-admin avec plusieurs agences	Il voit seulement les siennes et en choisit une
Non-admin avec une seule agence	Il entre directement dans cette agence
Non-admin sans agence	Message « Contactez l'administrateur »

Règles de sécurité de la modification d'un utilisateur (étape 8)

- Un administrateur ne peut pas modifier son propre rôle ni désactiver son propre compte.
- Le logiciel refuse de désactiver ou de rétrograder **le dernier administrateur actif** (contrôle fait dans la base, dans une transaction).
- Un utilisateur désactivé ne peut plus se connecter, mais n'est pas supprimé : ses opérations passées resteront rattachées à son nom.
- L'identifiant n'est pas modifiable (il sert à la connexion).
- Les mots de passe sont toujours hachés (bcrypt), minimum 8 caractères à la création et à la réinitialisation.
- Le bouton Enregistrer reste grisé tant que les agences ne sont pas chargées, pour ne pas effacer des affiliations par erreur.

5. Base de données

Version	Contenu	État
1	Table utilisateurs	En place
2	Tables agences et utilisateur_agences	En place
3	Colonne description dans agences	En place (migration avec _colonneExiste)
4 (prévu)	Tables roles, permissions et role_permissions	À faire

Table	Colonnes principales
`utilisateurs`	id, nom_complet, identifiant (unique), mot_de_passe_hash, role, actif, cree_le
`agences`	id, nom (unique), ville, telephone, description, actif, cree_le
`utilisateur_agences`	utilisateur_id, agence_id (clé primaire double, suppression en cascade)

Principes retenus : migrations qui vérifient avant d'agir ; requêtes avec paramètres (?) ; transactions pour les écritures liées (création et modification d'un utilisateur avec ses agences) ; LEFT JOIN pour conserver les utilisateurs sans agence dans la liste ; GROUP_CONCAT pour afficher les agences d'un utilisateur.

6. Fichiers ajoutés ou modifiés (dossier lib/)

Fichier	Rôle
utilisateur.dart	Rôles (rolesDisponibles, libelleRole), Utilisateur, Agence, UtilisateurResume, Session
database_service.dart	Base SQLite : migrations, utilisateurs, agences, affiliations, modification, mot de passe

Fichier	Rôle
apres_connexion_page.dart	Décide entre création d'agence, choix d'agence et accueil
creer_agence_page.dart	Formulaire d'agence avec zone de texte (description, 3 à 5 lignes, 250 caractères)
choix_agence_page.dart	Liste des agences accessibles, bouton Nouvelle agence pour l'admin
utilisateurs_page.dart	Liste des utilisateurs (rôle, agences, pastille Désactivé) ; clic pour modifier
creer_utilisateur_page.dart	Création : nom, identifiant, mot de passe, rôle, cases des agences
modifier_utilisateur_page.dart	Modification, interrupteur Compte actif, boîte de réinitialisation du mot de passe
home_page.dart	Accueil provisoire : pastilles rôle et agence, boutons Utilisateurs, Changer d'agence, Déconnexion

7. Conception du système de droits (à construire)

Besoin exprimé : l'administrateur doit pouvoir choisir lui-même ce que chaque rôle peut faire, parce que les fonctionnalités du logiciel s'ajouteront avec le temps.

Principe

	Qui décide	Où c'est stocké
Catalogue des fonctions (créer une vente, voir les prix d'achat...)	Le développeur	Dans le code
Qui a le droit d'utiliser chaque fonction	L'administrateur (grille de cases à cocher)	Dans la base de données

Exemples de codes de permission : `ventes.voir`, `ventes.creer`, `ventes.annuler`, `stock.voir`, `stock.modifier`, `achats.prix_achat_voir`, `rapports.voir`, `utilisateurs.gerer`, `agences.gerer`.

Grille de départ (tableau validé)

Fonction	Admin	Vendeur	Comptable	Magasinier
Caisse (ventes)	oui	oui	non	non
Stock et réceptions	oui	consultation	consultation	oui
Prix d'achat et marges	oui	non	oui	non
Rapports et finances	oui	non	oui	non
Utilisateurs et agences	oui	non	non	non

Fonctionnement et règles

- Une nouvelle fonction ajoutée au catalogue est insérée automatiquement dans la base au démarrage ; **par sécurité, personne ne l'a au départ sauf l'administrateur**, qui la coche ensuite pour les rôles voulus.
- Dans le code, un contrôle d'une ligne : `if (Session.peut('ventes.creer')) { ... }`.
- Le menu n'affiche que les entrées permises, **et** chaque opération sensible vérifie aussi le droit à l'intérieur de la fonction (cacher un bouton ne suffit pas).
- L'administrateur a toujours tous les droits et ne peut pas se les retirer.
- Les droits s'appliquent par rôle ; l'agence limite toujours où l'utilisateur travaille (affiliation existante).

Tables prévues (base en version 4)

Table	Contenu
`roles`	Code, libellé, indicateur « rôle système » (l'administrateur)
`permissions`	Catalogue des fonctions, synchronisé depuis le code au démarrage
`role_permissions`	Quelle permission est accordée à quel rôle

> Recommandation à confirmer : permettre à l'administrateur de **créer ses propres rôles** (par exemple caissier, gérant, livreur). La liste fixe rolesDisponibles du code serait alors remplacée par la table roles ; les utilisateurs existants gardent leur rôle.

8. Problèmes rencontrés et solutions

Problème	Solution
Erreur « duplicate column name: description » au démarrage	La colonne existait déjà alors que la base était en version 2. Migration qui teste l'existence avec _colonneExiste avant ALTER TABLE ADD COLUMN
Nouveaux rôles ajoutés à la main dans la liste déroulante	Les cases des agences n'apparaissaient que pour « vendeur ». Condition changée en « tout rôle sauf admin » et libellés centralisés dans libelleRole
Rechargement à chaud impossible après un gros changement	Arrêter avec q et relancer flutter run -d windows (ou R majuscule)
Image ou package ajouté non pris en compte	Redémarrage complet nécessaire (assets et packages natifs)

9. À tester demain (étape 8)

#	Action	Résultat attendu
1	Ouvrir une fiche depuis la liste des utilisateurs	Fiche préremplie, identifiant grisé, agences cochées
2	Changer les agences, nom ou rôle puis enregistrer	La liste se met à jour

#	Action	Résultat attendu
3	Se connecter avec l'utilisateur modifié	Il voit ses nouvelles agences
4	Désactiver un compte puis tenter de se connecter	« Identifiant ou mot de passe incorrect » ; pastille Désactivé dans la liste
5	Réactiver le compte	La connexion refonctionne
6	Réinitialiser un mot de passe (mots différents, trop court, correct)	Messages d'erreur puis « Mot de passe réinitialisé »
7	Ancien puis nouveau mot de passe	Ancien refusé, nouveau accepté
8	Ouvrir sa propre fiche d'administrateur	Rôle et interrupteur grisés, avec message explicatif
9	Rétrograder ou désactiver le seul administrateur actif	Refusé : « Il doit rester au moins un administrateur actif »
10	Passer un utilisateur en administrateur	Les cases disparaissent ; « Toutes les agences » dans la liste

10. Éléments provisoires à retirer ou changer

- Bouton « Reconfigurer (test) » de la page de connexion : à retirer ou à réserver à l'administrateur.
- Titre « Gestion Provenderie » (connexion et main.dart) : à remplacer par le nom du logiciel.
- Page d'accueil provisoire : à remplacer par un menu selon les droits.
- Les boutons Utilisateurs sont seulement masqués pour les non-admins : le contrôle à l'intérieur des pages viendra avec les permissions.

11. Plan pour demain

Sous-étape	Contenu
8 (suite)	Valider les tests de la modification des utilisateurs
9a	Base en version 4 : tables roles, permissions, role_permissions ; catalogue de départ et droits issus du tableau ; synchronisation au démarrage
9b	Page « Droits des rôles » : grille de cases à cocher pour l'administrateur (et gestion des rôles si confirmée)
9c	Menu principal et accueil qui n'affichent que ce que le rôle permet ; contrôle des droits dans les fonctions

Questions en suspens

- L'administrateur peut-il **créer de nouveaux rôles** (recommandé) ou seulement régler les droits des quatre rôles existants ?
- Le **catalogue d'articles** est-il commun à toutes les agences (mêmes produits et prix, stock séparé par agence) ou propre à chaque agence ?

- Nom définitif du logiciel, en remplacement de « Gestion Provenderie ».

12. Code final des deux fichiers de base

utilisateur.dart

```
/// Les rôles disponibles : code enregistré dans la base -> texte affiché.
const Map<String, String> rolesDisponibles = {
  'vendeur': 'Vendeur',
  'comptable': 'Comptable',
  'magasinier': 'Magasinier',
  'admin': 'Administrateur',
};

/// Le texte à afficher pour un rôle (ou le code lui-même s'il est inconnu).
String libelleRole(String role) => rolesDisponibles[role] ?? role;

/// Un utilisateur du logiciel.
class Utilisateur {
  final int id;
  final String nomComplet;
  final String identifiant;
  final String role;

  const Utilisateur({
    required this.id,
    required this.nomComplet,
    required this.identifiant,
    required this.role,
  });

  bool get estAdmin => role == 'admin';
}

/// Une agence (un point de vente).
class Agence {
  final int id;
  final String nom;
  final String? ville;
  final String? telephone;
  final String? description;

  const Agence({
    required this.id,
    required this.nom,
    this.ville,
    this.telephone,
    this.description,
  });

  factory Agence.fromMap(Map<String, Object?> m) => Agence(
        id: m['id'] as int,
        nom: m['nom'] as String,
        ville: m['ville'] as String?,
        telephone: m['telephone'] as String?,
        description: m['description'] as String?,
      );
}

/// Une ligne de la liste des utilisateurs (avec les noms de ses agences).
class UtilisateurResume {
  final int id;
  final String nomComplet;
  final String identifiant;
  final String role;
  final bool actif;
  final String agences; // noms séparés par des virgules ('' si aucune)

  const UtilisateurResume({
    required this.id,
    required this.nomComplet,
    required this.identifiant,
```

```
    required this.role,
    required this.actif,
    required this.agences,
  });

  bool get estAdmin => role == 'admin';
}

/// Retient l'utilisateur et l'agence choisie tant que le logiciel est ouvert.
class Session {
  static Utilisateur? utilisateurCourant;
  static Agence? agenceCourante;

  static void deconnecter() {
    utilisateurCourant = null;
    agenceCourante = null;
  }
}
```

database_service.dart

```
import 'dart:io';
import 'package:bcrypt/bcrypt.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'config_service.dart';
import 'utilisateur.dart';

/// Gère la base de données SQLite.
class DatabaseService {
  static Database? _db;
  static bool _ffiInitialise = false;

  // Version 1 : utilisateurs. Version 2 : agences + affiliations.
  // Version 3 : colonne description dans agences.
  static const int _versionBase = 3;

  static Future<void> ouvrir(AppConfig config) async {
    if (config.type != 'sqlite') {
      throw Exception('Type de base non pris en charge : ${config.type}');
    }

    if (!_ffiInitialise) {
      sqfliteFfiInit();
      _ffiInitialise = true;
    }

    final dossier = File(config.cheminBase).parent;
    await dossier.create(recursive: true);

    _db = await databaseFactoryFfi.openDatabase(
      config.cheminBase,
      options: OpenDatabaseOptions(
        version: _versionBase,
        onConfigure: (db) async {
          await db.execute('PRAGMA foreign_keys = ON');
        },
        onCreate: (db, version) async {
          await _creerTableUtilisateurs(db);
          await _creerTablesAgences(db);
        },
        onUpgrade: (db, ancienneVersion, nouvelleVersion) async {
          if (ancienneVersion < 2) {
            await _creerTablesAgences(db);
          }
          if (!await _colonneExiste(db, 'agences', 'description')) {
            await db.execute('ALTER TABLE agences ADD COLUMN description TEXT');
          }
        },
      ),
    );
  }

  static Future<void> _creerTableUtilisateurs(Database db) async {
    await db.execute('''
```

```
    CREATE TABLE utilisateurs (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nom_complet TEXT NOT NULL,
      identifiant TEXT NOT NULL UNIQUE,
      mot_de_passe_hash TEXT NOT NULL,
      role TEXT NOT NULL DEFAULT 'vendeur',
      actif INTEGER NOT NULL DEFAULT 1,
      cree_le TEXT NOT NULL
    )
  ''');
}

static Future<void> _creerTablesAgences(Database db) async {
  await db.execute('''
    CREATE TABLE agences (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nom TEXT NOT NULL UNIQUE,
      ville TEXT,
      telephone TEXT,
      description TEXT,
      actif INTEGER NOT NULL DEFAULT 1,
      cree_le TEXT NOT NULL
    )
  ''');

  await db.execute('''
    CREATE TABLE utilisateur_agences (
      utilisateur_id INTEGER NOT NULL,
      agence_id INTEGER NOT NULL,
      PRIMARY KEY (utilisateur_id, agence_id),
      FOREIGN KEY (utilisateur_id) REFERENCES utilisateurs(id) ON DELETE CASCADE,
      FOREIGN KEY (agence_id) REFERENCES agences(id) ON DELETE CASCADE
    )
  ''');
}

// Vérifie si une colonne existe déjà dans une table
static Future<bool> _colonneExiste(
  Database db,
  String table,
  String colonne,
) async {
  final infos = await db.rawQuery('PRAGMA table_info($table)');
  return infos.any((c) => c['name'] == colonne);
}

static Database get db {
  final d = _db;
  if (d == null) throw StateError("La base de données n'est pas ouverte");
  return d;
}

// ---------- Utilisateurs ----------

static Future<bool> aDesUtilisateurs() async {
  final resultat = await db.rawQuery('SELECT COUNT(*) AS n FROM utilisateurs');
  final n = resultat.first['n'] as int;
  return n > 0;
}

static Future<int> creerUtilisateur({
  required String nomComplet,
  required String identifiant,
  required String motDePasse,
  required String role,
  List<int> agenceIds = const [],
}) async {
  final hash = BCrypt.hashpw(motDePasse, BCrypt.gensalt());

  // Transaction : soit tout est enregistré, soit rien
  return db.transaction((txn) async {
    final id = await txn.insert('utilisateurs', {
      'nom_complet': nomComplet,
      'identifiant': identifiant,
```

```
      'mot_de_passe_hash': hash,
      'role': role,
      'actif': 1,
      'cree_le': DateTime.now().toIso8601String(),
    });

    for (final agenceId in agenceIds) {
      await txn.insert('utilisateur_agences', {
        'utilisateur_id': id,
        'agence_id': agenceId,
      });
    }

    return id;
  });
}

static Future<Utilisateur?> verifierConnexion(
  String identifiant,
  String motDePasse,
) async {
  final lignes = await db.query(
    'utilisateurs',
    where: 'identifiant = ? AND actif = 1',
    whereArgs: [identifiant],
    limit: 1,
  );

  if (lignes.isEmpty) return null;

  final ligne = lignes.first;
  final hash = ligne['mot_de_passe_hash'] as String;

  if (!BCrypt.checkpw(motDePasse, hash)) return null;

  return Utilisateur(
    id: ligne['id'] as int,
    nomComplet: ligne['nom_complet'] as String,
    identifiant: ligne['identifiant'] as String,
    role: ligne['role'] as String,
  );
}

// Liste de tous les utilisateurs, avec le nom de leurs agences
static Future<List<UtilisateurResume>> listerUtilisateurs() async {
  final lignes = await db.rawQuery('''
    SELECT u.id, u.nom_complet, u.identifiant, u.role, u.actif,
           GROUP_CONCAT(a.nom, ', ') AS agences
    FROM utilisateurs u
    LEFT JOIN utilisateur_agences ua ON ua.utilisateur_id = u.id
    LEFT JOIN agences a ON a.id = ua.agence_id
    GROUP BY u.id
    ORDER BY u.nom_complet
  ''');

  return lignes
      .map((m) => UtilisateurResume(
            id: m['id'] as int,
            nomComplet: m['nom_complet'] as String,
            identifiant: m['identifiant'] as String,
            role: m['role'] as String,
            actif: (m['actif'] as int) == 1,
            agences: (m['agences'] as String?) ?? '',
          ))
      .toList();
}

// Les numéros des agences auxquelles cet utilisateur est affilié
static Future<Set<int>> agenceIdsDeUtilisateur(int utilisateurId) async {
  final lignes = await db.query(
    'utilisateur_agences',
    columns: ['agence_id'],
    where: 'utilisateur_id = ?',
    whereArgs: [utilisateurId],
```

```
    );
    return lignes.map((m) => m['agence_id'] as int).toSet();
  }

  static Future<void> modifierUtilisateur({
    required int id,
    required String nomComplet,
    required String role,
    required bool actif,
    required List<int> agenceIds,
  }) async {
    await db.transaction((txn) async {
      final actuel = await txn.query(
        'utilisateurs',
        columns: ['role', 'actif'],
        where: 'id = ?',
        whereArgs: [id],
        limit: 1,
      );
      if (actuel.isEmpty) throw Exception('Utilisateur introuvable');

      // Sécurité : il doit toujours rester un administrateur actif
      final etaitAdminActif =
          actuel.first['role'] == 'admin' && actuel.first['actif'] == 1;
      if (etaitAdminActif && (role != 'admin' || !actif)) {
        final r = await txn.rawQuery(
          "SELECT COUNT(*) AS n FROM utilisateurs WHERE role = 'admin' AND actif = 1",
        );
        if ((r.first['n'] as int) <= 1) {
          throw Exception('Il doit rester au moins un administrateur actif');
        }
      }

      await txn.update(
        'utilisateurs',
        {
          'nom_complet': nomComplet,
          'role': role,
          'actif': actif ? 1 : 0,
        },
        where: 'id = ?',
        whereArgs: [id],
      );

      // On remplace les affiliations (aucune pour un administrateur)
      await txn.delete(
        'utilisateur_agences',
        where: 'utilisateur_id = ?',
        whereArgs: [id],
      );
      if (role != 'admin') {
        for (final agenceId in agenceIds) {
          await txn.insert('utilisateur_agences', {
            'utilisateur_id': id,
            'agence_id': agenceId,
          });
        }
      }
    });
  }

  static Future<void> reinitialiserMotDePasse(
    int id,
    String nouveauMotDePasse,
  ) async {
    final hash = BCrypt.hashpw(nouveauMotDePasse, BCrypt.gensalt());
    await db.update(
      'utilisateurs',
      {'mot_de_passe_hash': hash},
      where: 'id = ?',
      whereArgs: [id],
    );
  }
```

```
  // ---------- Agences ----------

  // L'admin voit toutes les agences, les autres seulement les leurs.
  static Future<List<Agence>> agencesDisponibles(Utilisateur utilisateur) async {
    final List<Map<String, Object?>> lignes;

    if (utilisateur.estAdmin) {
      lignes = await db.query('agences', where: 'actif = 1', orderBy: 'nom');
    } else {
      lignes = await db.rawQuery('''
        SELECT a.* FROM agences a
        INNER JOIN utilisateur_agences ua ON ua.agence_id = a.id
        WHERE ua.utilisateur_id = ? AND a.actif = 1
        ORDER BY a.nom
      ''', [utilisateur.id]);
    }

    return lignes.map(Agence.fromMap).toList();
  }

  // Transforme un texte vide en null (rien à enregistrer)
  static String? _videEnNull(String? texte) =>
      (texte == null || texte.trim().isEmpty) ? null : texte.trim();

  static Future<Agence> creerAgence({
    required String nom,
    String? ville,
    String? telephone,
    String? description,
  }) async {
    final villeNettoyee = _videEnNull(ville);
    final telNettoye = _videEnNull(telephone);
    final descriptionNettoyee = _videEnNull(description);

    final id = await db.insert('agences', {
      'nom': nom,
      'ville': villeNettoyee,
      'telephone': telNettoye,
      'description': descriptionNettoyee,
      'actif': 1,
      'cree_le': DateTime.now().toIso8601String(),
    });

    return Agence(
      id: id,
      nom: nom,
      ville: villeNettoyee,
      telephone: telNettoye,
      description: descriptionNettoyee,
    );
  }
}
```